TP2 UPDATE:

I have finished working on all the UI so I just need to add trainers which shouldn't be too challenging. There will still be a gym and it is likely that it will end there. I just need to complete the collision check on the map and add the locations inside the buildings.

TP1 UPDATE:

The project is more challenging than I have expected so I am going to make a few changes. It will still be Generation 3 and the mechanics will still be the same. I am going to be less focused on the narrative and just get the character on the map. I am not too focused on the animations. So for now, my plan is just to get the bare bone mechanics completed in order to have a complete game.

Project Description:

For my term project, I aim to develop a Pokémon game set in Generation 3. The game will feature Pokémon from previous generations and will utilize the same mechanics as previous Pokémon games. The story will go up to the first gym or as much as I can accomplish. For those who are unfamiliar, Pokémon is a franchise where pocket monsters engage in turn-based battles and progress through the game by catching more and challenging other trainers.

Similar Projects:

The project will draw inspiration from Pokémon Emerald, my favorite Pokémon game. In Pokémon Emerald, players control a young Pokémon Trainer who moves to the Hoenn region, and they are able to choose their gender and name. The journey begins in Littleroot, where the player, after setting up their character, must save the professor by selecting a starter Pokémon—Treecko, Torchic, or Mudkip. The player embarks on a quest to become the champion, battling trainers, capturing Pokémon, and thwarting the plans of the evil teams, Team Aqua and Team Magma. The ultimate goal is to collect gym badges, defeat the Elite 4, and defeat the current champion. My game will be based on this structure, with significant modifications to the story for a fresh and original experience. Set a decade after Pokémon Emerald, the protagonist will be the child of the original game's protagonist. The story, gym types, gym leaders, and the introduction of a new evil team will distinguish my game and Pokemon Emerald while retaining similar mechanics.

Structural Plan:

I will organize my files by grouping sprites for buildings, Pokémon, backgrounds, and characters. Multiple classes will be created for the game, including a player class and a Pokémon class. This approach will improve efficiency and allows us to easily track the pokemon and money of the player.

Algorithmic Plan:

The most challenging aspect of the project lies in the battle mechanics. The algorithm must determine the opponent's move, establish the order of Pokémon turns, calculate damage dealt, and allocate experience points for defeated Pokémon. Additionally, the system should

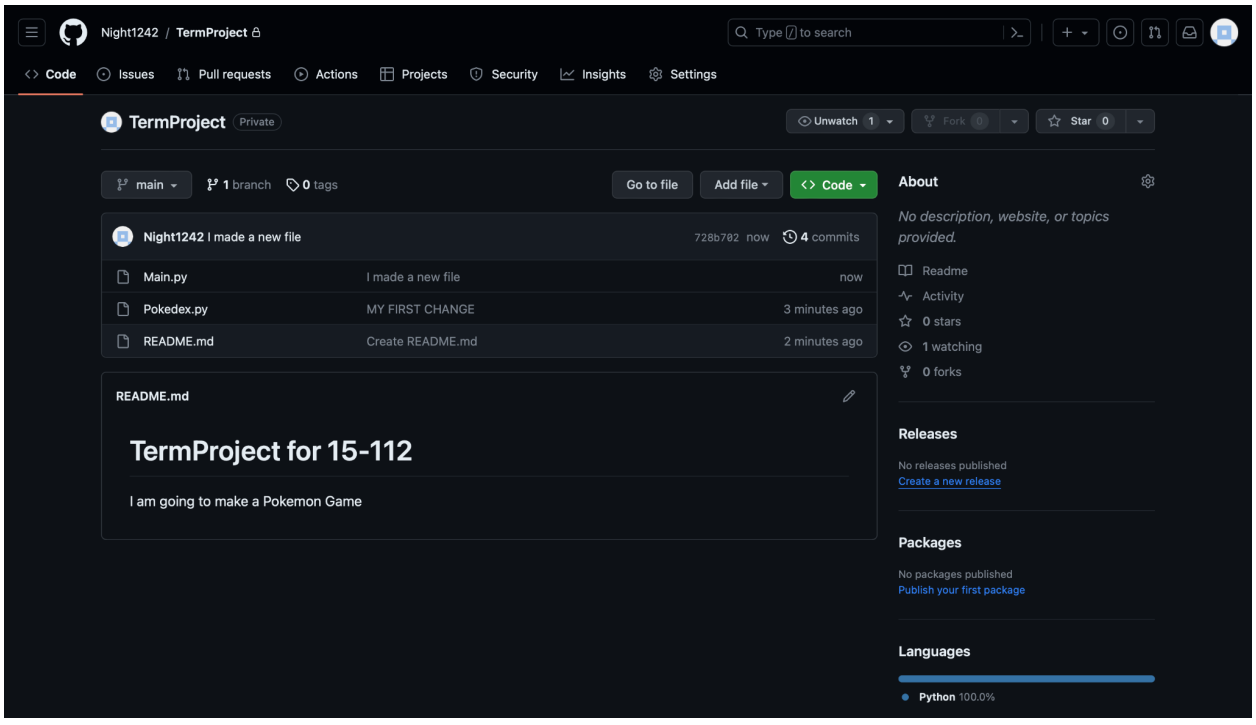
allow for Pokémon switching and item usage during battles. Creating effective and original classes will be important in streamlining the battle process.

Timeline Plan:

With a busy schedule in the coming weeks, I plan to complete the class and battle system by TP1 (November 27). While it doesn't need to be perfect, it should be functional and it will be refined throughout the week. By November 29, players should encounter wild Pokémon randomly when walking on grass, and most of the map should be completed. By December 1, the entire map, along with trainer placements and background music for overworld and battle scenarios, should be finalized. Until December 6, the focus will be on refining and polishing the game for a better presentation.

Version Control Plan:

For version control, I will utilize GitHub and Git. The Git repository contains my term project code, allowing me to push completed work to GitHub, stored in the cloud. This makes it easier to access recent or specific versions of the code repository, and I can add messages that can provide insights into the changes made. This ensures that everything is safe and sound just by committing on VS Code.



Module List:

I am planning to use numpy, PIL (image), playsound, pygame, and y pokedex in my project.